Practical 13

Generics & Reflection

Part A — Practice Programs (Hour 1)

1) Generic Containers — Box<T>, Pair<A,B> and a generic method

Code

```
public class GenericContainers {
    static class Box<T> {
        private T value;
        Box(T value) { this.value = value; }
        T get() { return value; }
        void set(T value) { this.value = value; }
    }

    static class Pair<A, B> {
        private final A first;
        private final B second;
        Pair(A first, B second) { this.first = first; this.second = second; }
        A getFirst() { return first; }
        B getSecond() { return second; }
        public String toString() { return "(" + first + ", " + second + ")"; }
    }

    static <T> T first(T[] array) {
        return array[0];
    }

    public static void main(String[] args) {
        Box<Integer> intBox = new Box<>(42);
        Box<String> strBox = new Box<>("hello");
        int n = intBox.get();            // no cast needed
        String s = strBox.get();         // no cast needed
        System.out.println("intBox = " + n + ", strBox = " + s);
        intBox.set(100);
        System.out.println("after set: " + intBox.get());
        // intBox.set("text");           // compile-time error: type safety

        Pair<String, Integer> p1 = new Pair<>("Riya", 88);
        Pair<Double, Boolean> p2 = new Pair<>(3.14, true);
        System.out.println("pairs: " + p1 + " and " + p2);
        System.out.println("p1 name = " + p1.getFirst() + ", marks = " + p1.getSecond());

        Integer[] nums = {7, 8, 9};
        String[] words = {"alpha", "beta"};
        System.out.println("first(nums) = " + first(nums) + ", first(words) = " + first(words));
    }
}
```

Output

```
intBox = 42, strBox = hello
after set: 100
pairs: (Riya, 88) and (3.14, true)
p1 name = Riya, marks = 88
first(nums) = 7, first(words) = alpha
```

2) Bounded Stats — <N extends Number> double average(N[])

Code

```
public class BoundedStats {
    static <N extends Number> double average(N[] values) {
        double sum = 0;
        for (N v : values) sum += v.doubleValue();
        return sum / values.length;
    }

    public static void main(String[] args) {
        System.out.println("Integer[] average: " + average(new Integer[] {1, 2, 3, 4}));
        System.out.println("Double[]  average: " + average(new Double[] {1.5, 2.5, 3.5}));
        System.out.println("Long[]    average: " + average(new Long[] {100L, 200L, 300L}));
        // average(new String[] {"a", "b"});   // compile error: String is not a Number (bound N
            extends Number)
        System.out.println("average(new String[]{...}) is rejected at compile time: String does not
            extend Number.");
    }
}
```

Output

```
Integer[] average: 2.5
Double[]  average: 2.5
Long[]    average: 200.0
average(new String[]{...}) is rejected at compile time: String does not extend Number.
```

A String[] is refused because the bound N extends Number is checked at compile time.

3) Generic Stack (For Advanced Learners — home) — array-backed, reused for two types

Code

```
import java.util.Optional;

public class GenericStack<T> {
    private T[] items;
    private int size = 0;

    @SuppressWarnings("unchecked")
    public GenericStack(int capacity) {
        items = (T[]) new Object[capacity];
    }

    public void push(T item) {
        if (size == items.length) {
            items = java.util.Arrays.copyOf(items, size * 2);   // grow when full
        }
        items[size++] = item;
    }

    /** Empty case handled with Optional instead of an exception or null. */
    public Optional<T> pop() {
        if (size == 0) return Optional.empty();
        T item = items[--size];
        items[size] = null;
        return Optional.of(item);
    }

    public boolean isEmpty() { return size == 0; }

    public static void main(String[] args) {
        GenericStack<Integer> ints = new GenericStack<>(2);
        ints.push(10); ints.push(20); ints.push(30);      // triggers growth
        System.out.println("pop: " + ints.pop().get() + ", " + ints.pop().get() + ", " +
            ints.pop().get());
        System.out.println("pop on empty stack: " + ints.pop());

        GenericStack<String> words = new GenericStack<>(4);
        words.push("one"); words.push("two");
        System.out.println("pop: " + words.pop().orElse("none") + ", " + words.pop().orElse("none")
            + ", "
                + words.pop().orElse("none (stack empty)"));
    }
}
```

Output

```
pop: 30, 20, 10
pop on empty stack: Optional.empty
pop: two, one, none (stack empty)
```

Empty case handled with Optional: pop() returns Optional.empty() instead of throwing or returning null.

Part B — Project Work (Hour 2): MiniBank

One reusable Repository<T> stores any type and finds each object's key by reflecting on the @Id annotation from Practical 7. Customer now carries @Id so the same class works for both Account and Customer. Files not listed are unchanged from Practical 12.

Customer — @Id on customerId (supplementary: Repository<Customer>)

Code

`model/Customer.java`

```
package model;

import model.annotation.Id;

import java.io.Serializable;

public class Customer implements Serializable {
    private static final long serialVersionUID = 1L;

    @Id
    private final String customerId;
    private String name;
    private String email;

    public Customer(String customerId, String name, String email) {
        this.customerId = customerId;
        this.name = name;
        this.email = email;
    }

    public String getCustomerId() { return customerId; }
    public String getName() { return name; }
    public String getEmail() { return email; }

    @Override
    public String toString() {
        return "Customer[" + customerId + ", " + name + ", " + email + "]";
    }
}
```

1-4) Repository<T> — save/findById/findAll/delete, @Id by reflection, @SafeVarargs saveAll, bounded average()

Code

service/Repository.java

```
package service;

import model.annotation.Id;

import java.lang.reflect.Field;
import java.util.ArrayList;
import java.util.HashMap;

public class Repository<T> {
    private final HashMap<String, T> store = new HashMap<>();

    /** Finds the field marked @Id by reflection and uses its value as the key. */
    private String idOf(T entity) {
        for (Field f : entity.getClass().getDeclaredFields()) {
            if (f.isAnnotationPresent(Id.class)) {
                try {
                    f.setAccessible(true);
                    return String.valueOf(f.get(entity));
                } catch (IllegalAccessException e) {
                    throw new IllegalStateException("Cannot read @Id field", e);
                }
            }
        }
        throw new IllegalArgumentException(entity.getClass().getSimpleName() + " has no @Id field");
    }

    public T save(T entity) {
        store.put(idOf(entity), entity);
        return entity;
    }

    public T findById(String id) {
        return store.get(id);
    }

    public ArrayList<T> findAll() {
```

```
        return new ArrayList<>(store.values());
    }

    public boolean delete(String id) {
        return store.remove(id) != null;
    }

    // supplementary: number of stored items
    public int count() {
        return store.size();
    }

    @SafeVarargs
    public final void saveAll(T... items) {
        for (T item : items) {
            save(item);
        }
    }

    /** Bounded type parameter: only Number subclasses are accepted. */
    public static <N extends Number> double average(N[] values) {
        double sum = 0;
        for (N v : values) {
            sum += v.doubleValue();
        }
        return values.length == 0 ? 0 : sum / values.length;
    }
}
```

Supplementary problem covered: count() returns the number of stored items.

5) main — Repository<Account> and Repository<Customer>

Code

service/MiniBank.java

```
package service;

import model.Account;
import model.Customer;

public class MiniBank {
    public static void main(String[] args) {
        Repository<Account> accounts = new Repository<>();
        accounts.save(new Account("AC0001", 5000, 0.04));
        accounts.saveAll(new Account("AC0002", 2000, 0.04), new Account("AC0003", 9000, 0.04));

        System.out.println("--- findById(\"AC0002\") ---");
        System.out.println(accounts.findById("AC0002"));
        System.out.println("findById(\"AC0099\") = " + accounts.findById("AC0099"));

        System.out.println("--- findAll() ---");
        accounts.findAll().stream().sorted().forEach(System.out::println);
        System.out.println("count = " + accounts.count());

        System.out.println("--- delete(\"AC0001\") ---");
        System.out.println("deleted: " + accounts.delete("AC0001") + ", deleting again: " +
            accounts.delete("AC0001"));
        System.out.println("count = " + accounts.count());

        System.out.println("--- Same Repository class for Customer ---");
        Repository<Customer> customers = new Repository<>();
        customers.saveAll(new Customer("CUST101", "Riya", "riya@example.com"),
                          new Customer("CUST102", "Aman", "aman@example.com"));
        System.out.println(customers.findById("CUST102"));
        System.out.println("customers stored: " + customers.count());

        System.out.println("--- Bounded generic method average() ---");
        System.out.println("Integer[]: " + Repository.average(new Integer[] {4, 8, 12}));
        System.out.println("Double[] : " + Repository.average(new Double[] {2.5, 3.5}));
        System.out.println("Long[]   : " + Repository.average(new Long[] {10L, 20L, 40L}));

        System.out.println("--- Entity without @Id ---");
        try {
            new Repository<String>().save("no id here");
        } catch (IllegalArgumentException e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}
```

Output

```
--- findById("AC0002") ---
Account[AC0002, balance=2000]
findById("AC0099") = null
--- findAll() ---
Account[AC0001, balance=5000]
Account[AC0002, balance=2000]
Account[AC0003, balance=9000]
count = 3
--- delete("AC0001") ---
deleted: true, deleting again: false
count = 2
--- Same Repository class for Customer ---
Customer[CUST102, Aman, aman@example.com]
customers stored: 2
--- Bounded generic method average() ---
Integer[]: 8.0
Double[] : 3.0
Long[]   : 23.333333333333332
--- Entity without @Id ---
Error: String has no @Id field
```

Package layout, build and tag

Code

```
minibank/
 |-- exception/
 |   |-- AccountNotFoundException.java
 |   |-- BankException.java
 |   |-- DailyLimitExceededException.java
 |   |-- InsufficientFundsException.java
 |   +-- InvalidAmountException.java
 |-- model/
 |   |-- annotation/
 |   |   |-- Id.java
 |   |   |-- MaxLength.java
 |   |   +-- Positive.java
 |   |-- Account.java
 |   |-- Customer.java
 |   |-- InterestBearing.java
 |   |-- Premium.java
 |   |-- PremiumAccount.java
 |   +-- Transactable.java
 |-- service/
 |   |-- AccountStore.java
 |   |-- MiniBank.java
 |   |-- ReportGenerator.java
 |   |-- Repository.java
 |   |-- StatePersister.java
 |   |-- TransactionLog.java
 |   |-- TransactionProcessor.java
 |   +-- WithdrawRule.java
 +-- util/
     |-- AnnotationValidator.java
     +-- Validator.java

javac -d out $(find exception model util service -name "*.java")
java -cp out service.MiniBank
cd out
jar --create --file minibank.jar --main-class=service.MiniBank .
java -jar minibank.jar

cd ..
git add .
git commit -m "Practical 13 - generic repository"
git tag lab-13
```

The same Repository class works for Account and Customer without any change, and findById returns the correct object.

Key Questions / Analysis / Interpretation

1. What problem do generics solve compared with using Object?

With Object, a container accepts anything and every read needs a cast that can fail at run time with ClassCastException. Generics move that check to compile time: Repository<Account> only accepts Account and returns Account, so no casts are needed and mistakes are caught before the program runs.

2. What is a bounded type parameter, and why is <N extends Number> useful?

A bound restricts which types may be used for T. With <N extends Number>, the compiler only allows Integer, Double, Long and other Number subclasses, and inside the method it is safe to call Number methods such as

doubleValue(). Passing String[] is a compile error.

3. What is type erasure?

The compiler checks generic types and then erases them: Repository<Account> becomes a raw Repository in the byte code, with casts inserted where needed. So generic type information does not exist at run time, which is why new T(), new T[n] and instanceof List<String> are not allowed and why Repository finds the id field from the actual object (entity.getClass()) instead of from T.